

#1

```
SELECT airline_id,  
       UPPER(airline_name) AS name_upper  
FROM airline  
ORDER BY airline_id;
```

The screenshot shows a database IDE interface. The top panel displays a SQL query in a console window:

```
1 SELECT airline_id,  
2     UPPER(airline_name) AS name_upper  
3 FROM airline  
4 ORDER BY airline_id;
```

The bottom panel shows the results of the query in a table with 17 rows. The columns are labeled 'airline_id' and 'name_upper'. The data is as follows:

airline_id	name_upper
1	KAZAIR
2	KAZAIR
3	KAZAIR
4	GALLIAAIR
5	LISBOAWINGS
6	MAZOVIAAIR
7	GALLIAAIR
8	LISBOAWINGS
9	MAZOVIAAIR
10	GALLIAAIR
11	LISBOAWINGS
12	MAZOVIAAIR
13	AIR ASTANA
14	AIR ASTANA
15	AIR ASTANA
16	AIR ASTANA
17	AIR ASTANA

The interface also shows a 'Database Explorer' on the left with a tree view of the database structure, including 'lab2@localhost' and 'task2@localhost'. The bottom status bar indicates the current file is 'lab2.sql' and the query is executed in 'task2@localhost'.

#2

UPDATE airline

SET airline_name = *REPLACE*(airline_name, 'Air', 'Aero')

WHERE airline_name LIKE '%Air%';

SELECT airline_id, airline_name FROM airline ORDER BY airline_id;

The screenshot shows a database IDE interface with the following components:

- Database Explorer:** Shows a tree view of server objects including 'lab2@localhost [2]', 'lab2@localhost [3]', and 'task2@localhost'.
- Console:** Displays the executed SQL script:

```
1 UPDATE airline
2 SET airline_name = REPLACE(airline_name, 'Air', 'Aero')
3 WHERE airline_name LIKE '%Air%';
4 SELECT airline_id, airline_name FROM airline ORDER BY airline_id;
```
- Output:** Shows the results of the SELECT statement in a table with 17 rows. The columns are 'airline_id' and 'airline_name'. The results are as follows:

airline_id	airline_name
1	KazAero
2	KazAero
3	KazAero
4	GalliaAero
5	LisboaWings
6	MazoviaAero
7	GalliaAero
8	LisboaWings
9	MazoviaAero
10	GalliaAero
11	LisboaWings
12	MazoviaAero
13	Aero Astana
14	Aero Astana
15	Aero Astana
16	Aero Astana
17	Aero Astana

The bottom status bar indicates the file encoding is UTF-8, 4 spaces, and the trial version of the IDE.

#3

SELECT

f1.flight_id AS flight_id_airline1,

f2.flight_id AS flight_id_airline2,

f1.departing_airport_id,

f1.arriving_airport_id,

f1.sch_departure_time,

f1.sch_arrival_time

FROM flights f1

JOIN flights f2

ON f1.departing_airport_id = f2.departing_airport_id

AND f1.arriving_airport_id = f2.arriving_airport_id

AND f1.sch_departure_time = f2.sch_departure_time

AND f1.sch_arrival_time = f2.sch_arrival_time

WHERE f1.airline_id = 1

AND f2.airline_id = 2

ORDER BY f1.sch_departure_time, f1.departing_airport_id, f1.arriving_airport_id;

The screenshot shows a database IDE interface. The top panel displays a SQL query in a console window. The query is a SELECT statement joining two flight tables (f1 and f2) based on airport and schedule times, filtering for specific airlines, and ordering by departure time. The bottom panel shows the results of the query in a table format. The table has 5 columns: flight_id_airline1, flight_id_airline2, departing_airport_id, arriving_airport_id, and sch_departure_time. The results show 5 rows of data.

flight_id_airline1	flight_id_airline2	departing_airport_id	arriving_airport_id	sch_departure_time
1	7	1	2	2024-01-10 08:00:00
2	7	1	2	2024-01-10 08:00:00
3	7	1	2	2024-01-10 08:00:00
5	8	1	201	2024-02-10 08:00:00
6	8	1	201	2024-02-10 08:00:00

#4

SELECT *

FROM airport

WHERE airport_name ILIKE '%Reginal%'

AND airport_name ILIKE '%Air%'

ORDER BY airport_name;

The screenshot shows a database management tool interface. The central pane displays a SQL query:

```
1 SELECT *
2 FROM airport
3 WHERE airport_name ILIKE '%Reginal%'
4 AND airport_name ILIKE '%Air%'
5 ORDER BY airport_name;
6
```

The bottom pane shows the results of the query in a table with 4 rows. The columns are: airport_id, airport_name, country, state, city, created_at, and updated_at.

airport_id	airport_name	country	state	city	created_at	updated_at
284	Reginal Air Internati...	Country_284	State_284	City_284	2025-10-06 19:30:41.821796	2025-10-06 19:30:41.821796
286	Reginal Air Internati...	Country_284	State_284	City_284	2025-10-06 19:30:44.344095	2025-10-06 19:30:44.344095
285	Reginal Airport Air S...	Country_205	State_205	City_205	2025-10-06 19:30:41.821796	2025-10-06 19:30:41.821796
287	Reginal Airport Air S...	Country_205	State_205	City_205	2025-10-06 19:30:44.344095	2025-10-06 19:30:44.344095

The interface also includes a Database Explorer on the left, a Files pane on the right, and a Services pane at the bottom left. The bottom status bar shows the database console path: Database Consoles > task2@localhost > console [task2@localhost].

#5

SELECT DISTINCT

CONCAT_WS(' ', TRIM(first_name), TRIM(last_name)) AS passenger_name,

TO_CHAR(date_of_birth, 'FMMonth DD, YYYY') AS birth_date_pretty

FROM passengers

ORDER BY passenger_name;

The screenshot shows a database IDE interface. The top panel displays a SQL query in a console window. The query is as follows:

```
1 SELECT DISTINCT
2   CONCAT_WS(' ', TRIM(first_name), TRIM(last_name)) AS
3   TO_CHAR(date_of_birth, 'FMMonth DD, YYYY') AS
4 FROM passengers
5 ORDER BY passenger_name;
6
```

The bottom panel shows the results of the query in a table with 12 rows. The columns are 'passenger_name' and 'birth_date_pretty'.

passenger_name	birth_date_pretty
Aida Nazarova	June 25, 1993
Ali Ali	May 12, 1999
Anna Anna	March 08, 1995
Aruzhan Sadykova	April 10, 2001
Daniyar Samatov	November 30, 1998
Erkan Tursyn	January 15, 1990
Ivan Ivanov	December 12, 1988
John Smith	August 18, 1985
Madina Bekova	February 02, 1996
Nurlan Avezov	July 21, 1992
Oleg Petrov	September 05, 2000
Zhan Kalrat	October 17, 1997

The interface also shows a 'Database Explorer' on the left with a tree view of server objects, and a 'Services' section at the bottom left. The bottom status bar indicates the database is 'task2@localhost' and the console is 'console [task2@localhost]'.

#6

```
SELECT flight_id,  
  
       sch_arrival_time,  
  
       act_arrival_time,  
  
       (act_arrival_time - sch_arrival_time) AS arrival_delay  
  
FROM flights  
  
WHERE act_arrival_time > sch_arrival_time  
  
ORDER BY arrival_delay DESC, flight_id;
```

The screenshot shows a database IDE interface. The central pane displays a SQL query that selects flight details and calculates the arrival delay. The query is executed against a database named 'task2'. The results pane shows 7 rows of data, sorted by arrival delay in descending order. The columns are: flight_id, sch_arrival_time, act_arrival_time, and arrival_delay. The arrival delay is displayed in a human-readable format (years, months, days, hours, minutes, seconds).

flight_id	sch_arrival_time	act_arrival_time	arrival_delay
1	2024-01-10 12:00:00.000000	2024-01-10 12:10:00.000000	0 years 0 mons 0 days 0 hours 10 mins 0.0 secs
2	2024-01-10 12:00:00.000000	2024-01-10 12:10:00.000000	0 years 0 mons 0 days 0 hours 10 mins 0.0 secs
3	2024-01-10 12:00:00.000000	2024-01-10 12:10:00.000000	0 years 0 mons 0 days 0 hours 10 mins 0.0 secs
4	2024-02-10 12:00:00.000000	2024-02-10 12:10:00.000000	0 years 0 mons 0 days 0 hours 10 mins 0.0 secs
5	2024-02-10 12:00:00.000000	2024-02-10 12:10:00.000000	0 years 0 mons 0 days 0 hours 10 mins 0.0 secs
6	2024-02-10 12:00:00.000000	2024-02-10 12:10:00.000000	0 years 0 mons 0 days 0 hours 10 mins 0.0 secs
7	2024-01-10 12:00:00.000000	2024-01-10 12:10:00.000000	0 years 0 mons 0 days 0 hours 10 mins 0.0 secs

#7

SELECT DISTINCT

CONCAT_WS(' ', INITCAP(TRIM(first_name)), INITCAP(TRIM(last_name))) AS
passenger_name,

EXTRACT(YEAR FROM AGE(CURRENT_DATE, date_of_birth))::int AS age_years,

CASE

WHEN AGE(CURRENT_DATE, date_of_birth) BETWEEN INTERVAL '18 years' AND
INTERVAL '35 years' THEN 'Young'

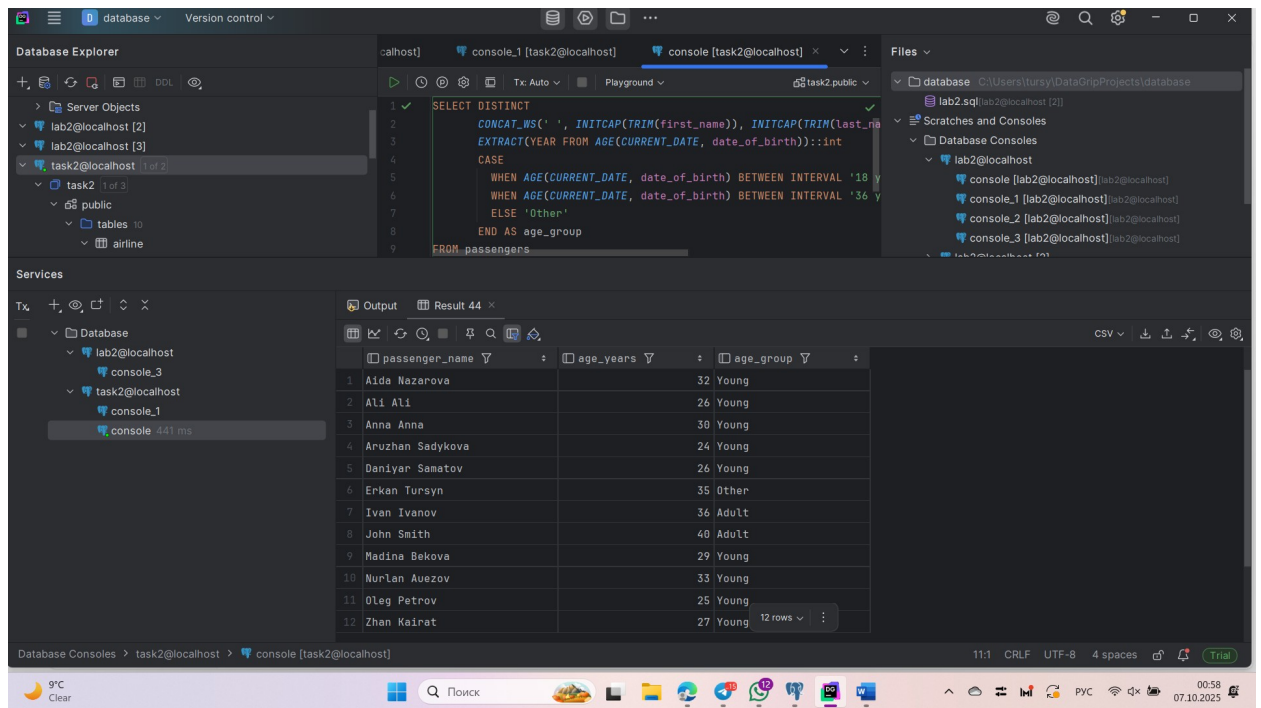
WHEN AGE(CURRENT_DATE, date_of_birth) BETWEEN INTERVAL '36 years' AND
INTERVAL '55 years' THEN 'Adult'

ELSE 'Other'

END AS age_group

FROM passengers

ORDER BY passenger_name;



#8

SELECT

ticket_price,

CASE

WHEN ticket_price < 20000 THEN 'Cheap'

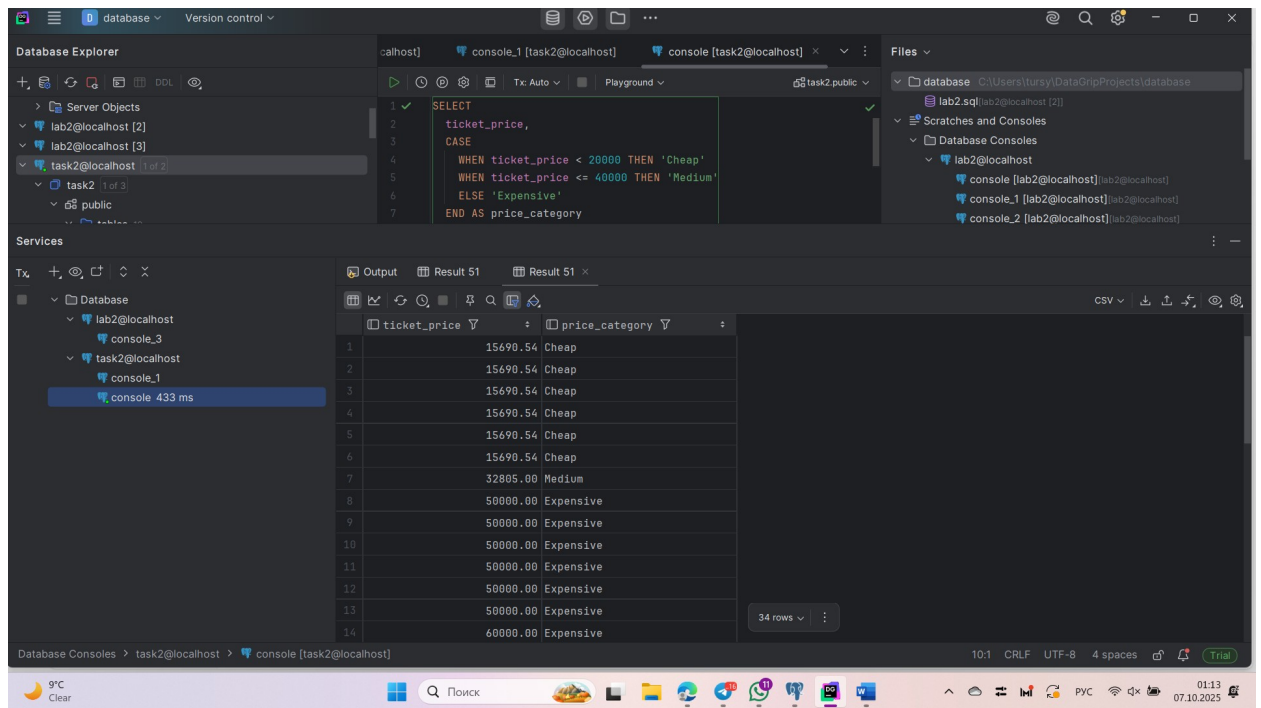
WHEN ticket_price <= 40000 THEN 'Medium'

ELSE 'Expensive'

END AS price_category

FROM booking

ORDER BY ticket_price;



#9

```
SELECT airline_country AS country,
       COUNT(*)        AS airline_count
FROM airline
GROUP BY airline_country
```

ORDER BY airline_count DESC, country;

The screenshot shows a database IDE interface. The central pane displays a SQL query:

```
1 SELECT airline_country AS country,  
2       COUNT(*) AS airline_count  
3 FROM airline  
4 GROUP BY airline_country  
5 ORDER BY airline_count DESC, country;  
6
```

The bottom pane shows the query results in a table with 5 rows:

country	airline_count
1 Kazakhstan	5
2 France	3
3 Poland	3
4 Portugal	3
5 Turkey	3

The interface also includes a Database Explorer on the left, a File Explorer on the right, and a Services pane at the bottom left. The status bar at the bottom indicates 6:1 CRLF UTF-8 4 spaces and a Trial version.

#10

SELECT

flight_id,

```
sch_arrival_time,  
act_arrival_time,  
(act_arrival_time - sch_arrival_time) AS arrival_delay  
FROM flights  
WHERE act_arrival_time IS NOT NULL  
AND sch_arrival_time IS NOT NULL  
AND act_arrival_time > sch_arrival_time  
ORDER BY arrival_delay DESC, flight_id;
```

The screenshot shows a database IDE with a SQL query in the center pane and its results in the bottom pane. The query is as follows:

```
1 SELECT  
2 flight_id,  
3 sch_arrival_time,  
4 act_arrival_time,  
5 (act_arrival_time - sch_arrival_time) AS arrival_delay  
6 FROM flights  
7 WHERE act_arrival_time IS NOT NULL
```

The results pane shows 7 rows of data. The columns are flight_id, sch_arrival_time, act_arrival_time, and arrival_delay. The arrival_delay is calculated as the difference between act_arrival_time and sch_arrival_time.

flight_id	sch_arrival_time	act_arrival_time	arrival_delay
1	2024-01-10 12:00:00.000000	2024-01-10 12:10:00.000000	0 years 0 mons 0 days 0 hours 10 mins 0.0 secs
2	2024-01-10 12:00:00.000000	2024-01-10 12:10:00.000000	0 years 0 mons 0 days 0 hours 10 mins 0.0 secs
3	2024-01-10 12:00:00.000000	2024-01-10 12:10:00.000000	0 years 0 mons 0 days 0 hours 10 mins 0.0 secs
4	2024-02-10 12:00:00.000000	2024-02-10 12:10:00.000000	0 years 0 mons 0 days 0 hours 10 mins 0.0 secs
5	2024-02-10 12:00:00.000000	2024-02-10 12:10:00.000000	0 years 0 mons 0 days 0 hours 10 mins 0.0 secs
6	2024-01-10 12:00:00.000000	2024-01-10 12:10:00.000000	0 years 0 mons 0 days 0 hours 10 mins 0.0 secs
7	2024-02-10 12:00:00.000000	2024-02-10 12:10:00.000000	0 years 0 mons 0 days 0 hours 10 mins 0.0 secs

